

CS112: Monday 21 September, 12:15 to 1:20

DRAFT, RECONSTRUCTED FROM COURSE-MAP.YML AND THE TEACHING LOG

Friday we laid a grid out in memory and found that `grid[r][c]` is one address plus `r` times the row width plus `c`, which is why a function has to be told how wide a row is.

Today: you can make a type. Week 0 handed you `int` and `double` and that was the list. From now on the list is as long as you like, and the first one is a `Pair`. The interesting part is not the syntax, it is that you get to decide what other people are allowed to know about it.

TIME	BEAT
12:20	1. A class is a new type
12:35	2. <code>Pair.h</code> says what, <code>Pair.cpp</code> says how
12:52	3. <code>public</code> and <code>private</code>
13:08	4. <code>const</code> methods

Beat 1. A class is a new type (15 min)

```
Pair p(3, 4);
cout << p.getFirst() << endl;
```

Declared, passed, returned, exactly like an `int`. The difference is that you wrote what it means.

Beat 2. `Pair.h` says what, `Pair.cpp` says how (17 min)

The same split as week 1's `dice.h` and `dice.cpp`, so nothing here is new except the word `class`.

```
// Pair.h
class Pair {
public:
    Pair(Item f, Item s);
    Item getFirst() const;
private:
    Item myFirst, mySecond;
};

// Pair.cpp
Item Pair::getFirst() const { return myFirst; }
```

Pair:: is what binds them. Without it you have written a free function that happens to share a name, and the compiler will tell you so in a way that takes ten minutes to decode the first time.

Beat 3. `public` and `private` (16 min)

`public` is your promise to everyone else. `private` is the freedom to change your mind later.

Talk to your neighbor. You change `myFirst` and `mySecond` to store the pair some completely different way. Whose code has to be rewritten?

Only yours, as long as the public methods still behave. That is the entire argument for `private`, and it is worth the minute it takes to land.

Beat 4. `const` methods (12 min)

```
Item getFirst() const;           // const goes AFTER the parameter
list
```

It promises this method does not change the object. A getter can make that promise; a setter cannot, because changing the object is its job. And only `const` methods may be called on a `const` object, which is how the promise starts mattering to code you did not write.